Policy Summarizer: An AI Agent for Summarizing Policy Documents

Bavishyaa | Project write-up | AI product management portfolio

Overview

Policy Summarizer is a Python agent, built and run in Google Colab, that reads a policy from a PDF, a text file or a web page and returns a summary. It sends the text to a Google Gemini model through the google-genai SDK, with an optional focus topic that tells the model what to pay special attention to.

Problem and Users

Policies in HR, compliance, insurance, vendor management and IT security are long, dense and read inconsistently. People skim them, miss deadlines and exceptions, and compliance teams spend time re-explaining the same rules.

- **Employees and new joiners** need a fast first-pass understanding of what applies to them.
- **Analysts and compliance reviewers** need consistent summaries across many policies.
- **Managers** need a quick brief before explaining a policy to a team.

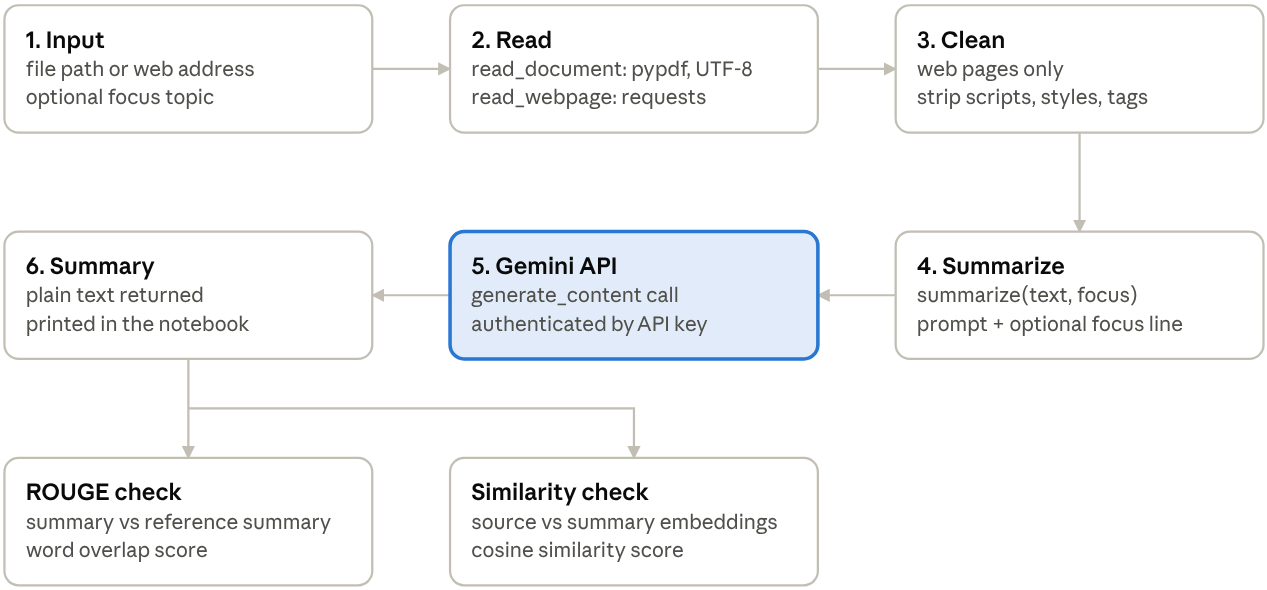
The value is faster first-pass understanding, support for web pages as well as files, and an optional focus mode for a specific topic such as maternity leave or data retention.

Proposed success metrics: time to understand a policy compared with reading the full text, faithfulness of the summary to the source, coverage of obligations and deadlines, user trust rating, and cost per document.

Architecture

The agent is one Python module, `summarizer.py`, written from a Colab notebook cell with `%%writefile`. It has three functions: `read_document`, `read_webpage` and `summarize`. Two further notebook cells handle evaluation.

Two readers feed one summarize step, and two checks score the result



architecture · read, summarize, evaluate

Text comes in through one of two readers, is cleaned if it came from the web, and goes to the Gemini model with an optional focus topic. Two evaluation cells then score the summary against a reference summary and against the source.

1. **Configuration.** The google-genai client is created at the top of the module with an API key. The key belongs in Colab Secrets or an environment variable, not typed into the notebook.
2. **Reading a document (`read_document`).** PDFs are read page by page with pypdf; other files are opened as UTF-8 text.
3. **Reading a web page (`read_webpage`).** The page is downloaded with `requests` using a browser-style User-Agent header, and a failed request raises an error.
4. **Cleaning (inside `read_webpage`).** Regular expressions remove script blocks, style blocks and HTML tags, and whitespace is collapsed into plain text.
5. **Summarizing (`summarize`).** The text is placed in a prompt that asks the model to summarize the policy document. If a focus topic is given, a line asking for special attention to it is added. The prompt goes to the Gemini model through `generate_content`, and the response text is returned.
6. **Evaluation.** A ROUGE cell compares the summary with a reference summary, and an embedding cell computes the cosine similarity between the source and the summary.

Use it from a notebook cell: get the text with `read_document(path)` or `read_webpage(url)`, then call `summarize(text)`, or `summarize(text, focus="maternity leave")` to steer the summary.

Prompt Design

- **Simple instruction.** The prompt asks the model to summarize the policy document and places the full text after it. This is a deliberately minimal first version.
- **Focus parameter.** A focus topic adds a line asking for special attention to it, so the user can steer emphasis without editing the prompt.
- **What is not constrained yet.** The prompt does not tell the model to use only the document, to follow a fixed structure, or to flag information that is missing. Adding those instructions is the next prompt improvement, because they target hallucination and make summaries comparable across documents.
- **Full text in context.** The whole document goes to the model in one call. That keeps the design simple but limits it to documents that fit in the model's context window.

Known Limitations and Mitigations

- **Very long documents** may exceed the context window. Mitigation: split into chunks, summarize each, then summarize the summaries.
- **Scanned PDFs** have no text layer. Mitigation: add OCR.
- **Layout loss.** pypdf can extract tables and multi-column pages out of order. Mitigation: use a layout-aware parser for complex documents.
- **Basic web cleaning.** Regular expressions remove tags, but menus and footers on a page stay in the text. Mitigation: use an HTML parser to extract the main content.
- **No source citations.** Mitigation: return page or section references with each key point.
- **Unconstrained output.** Without grounding instructions the model can add or omit details. Mitigation: tighten the prompt and evaluate against a test set.
- **Not legal or compliance advice.** A person should verify high-stakes points against the source.
- **Secrets and privacy.** The API key must not be stored in the notebook or shown in screenshots, and confidential policies should only go through an approved service under the organization's data-handling terms.

Evaluation Approach

Two automatic checks are implemented in the notebook. A broader evaluation is planned and has not been run yet.

- **ROUGE (implemented).** Compares the generated summary with a human-written reference summary by word overlap. It is a quick regression check after prompt changes, but it cannot tell whether a number or deadline is correct.
- **Embedding similarity (implemented).** Computes the cosine similarity between the embedding of the source document and the embedding of the summary. A high score suggests the summary stays on topic, but it does not prove each fact is right.
- **Planned: faithfulness and coverage.** Check that every claim in the summary is supported by the source, track whether key facts (deadlines, penalties, exceptions) are covered, and confirm that dates and amounts in the summary appear in the source.
- **Planned: test set and human review.** Score [N] policies from several domains against reference summaries, review a sample against a written rubric, and track cost and latency per document.

Results: [to be added after the evaluation run].

Roadmap

1. Chunked summarisation for long documents.
2. Page and section citations on every key point.
3. Follow-up question answering over the document, using retrieval.
4. Comparison of two policy versions, highlighting what changed.
5. OCR support for scanned documents.
6. A web interface with logging and feedback capture.
7. A human review step for high-stakes outputs.

Product Perspective

- **Accuracy versus brevity.** Short summaries are easier to read but can drop exceptions, so the next prompt version should ask for deadlines, penalties and exceptions explicitly.
- **Automation versus trust.** The tool is a first-pass aid. Citations are planned so users can verify claims against the source quickly.

- **Cost and latency versus quality.** The model is a single parameter in the call, so a team can trade capability for speed or cost per use case.
- **Domain portability.** The same functions serve HR, insurance, IT security and vendor documents. Only the prompt wording and the evaluation set change.

Technology Stack

Python, Google Colab, Google Gemini API (google-genai SDK), pypdf, requests, ROUGE scoring and embedding-based cosine similarity.